

Construção de Software

Encontro 5

Exceções e tratamento de erros; Manipulação de exceções

Objetivos do Encontro

- Apresentar conceitos centrais sobre **Exceções e tratamento de erros; Manipulação de exceções.**
- Discutir exemplos práticos com UML e código.
- Fixar conteúdos com exercícios práticos.

Exceções

- **Exceção** é um evento anômalo que interrompe o fluxo normal de execução do programa
- Pode ser causada por erros de programação, problemas de entrada/saída, falhas de rede, entre outros
- Exceções podem ser tratadas ou propagadas para o chamador

Orientação a objetos e Exceções

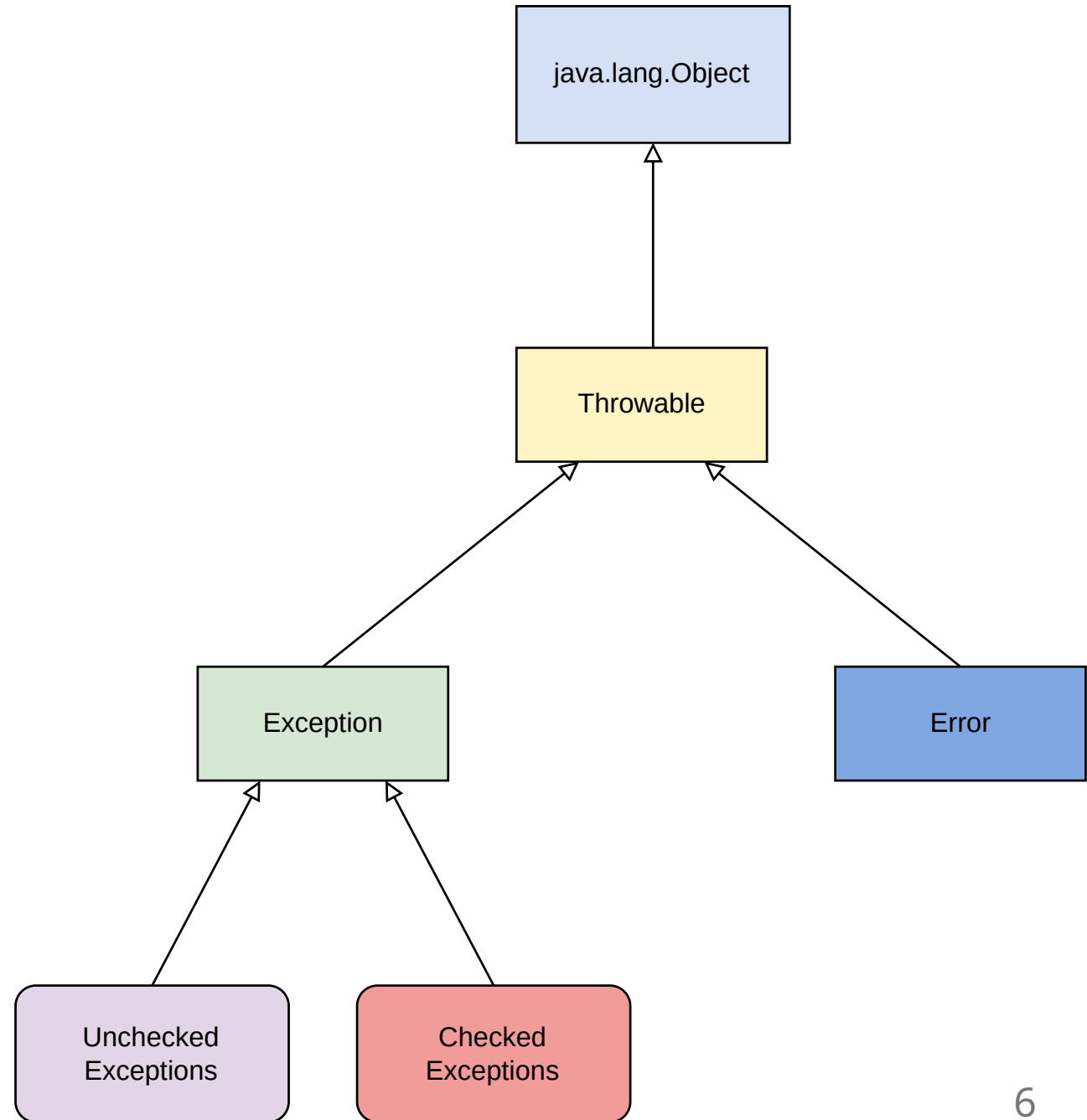
Polimorfismo e herança são conceitos fundamentais na orientação a objetos que trazem benefícios significativos para o tratamento de exceções:

- Em Java, exceções são representadas por objetos que estendem a classe `Throwable`
- Existem duas categorias principais: **checked exceptions** (verificadas em tempo de compilação) e **unchecked exceptions** (não verificadas, como `RuntimeException`)
- O tratamento de exceções é feito com os blocos `try`, `catch`, `finally` e a palavra-chave `throw`
- Permite criar código mais robusto e resiliente a falhas
- Facilita a separação entre lógica de negócio e tratamento de erros

Tipos de exceções

- **Checked Exceptions:** devem ser tratadas obrigatoriamente.
- **Unchecked Exceptions:** não precisam ser declaradas.

Hierarquia de Exceções



- **RuntimeException:** Pai de todas as exceções não verificadas, como NullPointerException, IllegalArgumentException, etc.
- **Exception:** Pai de todas as exceções verificadas, como IOException, SQLException,
- **Error:** problemas graves do sistema que não devem ser tratados (ex: OutOfMemoryError)

Erros vs Exceções

Definições:

- **Erro** é um problema sério que ocorre durante a execução do programa e geralmente não pode ser recuperado (ex: falta de memória).
- **Exceção** é uma condição anômala que pode ser tratada pelo programa (ex: arquivo não encontrado).
- Erros indicam falhas no ambiente de execução, enquanto exceções indicam problemas na lógica do programa.

Error

O **Error** é uma subclasse de Throwable que indica problemas graves do sistema que geralmente não podem ser tratados ou recuperados. Exemplos comuns incluem:

- OutOfMemoryError: quando a JVM fica sem memória
- StackOverflowError: quando a pilha de chamadas excede seu limite
- VirtualMachineError: erros relacionados à máquina virtual Java

Exemplo prático de Error

```
public class ExemploError {  
    public static void main(String[] args) {  
        // Exemplo de StackOverflowError  
        try {  
            metodoRecurativo();  
        } catch (StackOverflowError e) {  
            System.out.println("StackOverflowError capturado: " + e.getMessage());  
        }  
    }  
    public static void metodoRecurativo() {  
        metodoRecurativo(); // Chamada recursiva infinita  
    }  
}
```

RuntimeException

Definições:

- **RuntimeException** é a superclasse de todas as exceções que podem ser lançadas durante a execução normal do programa
- São **unchecked exceptions**, ou seja, não precisam ser declaradas ou capturadas
- Exemplos comuns: `NullPointerException`, `IllegalArgumentException`, `ArrayIndexOutOfBoundsException`
- Geralmente indicam erros de programação, como acesso a referências nulas ou índices inválidos
- Devem ser evitadas com boas práticas de codificação e validação de entrada
- Podem ser lançadas explicitamente com `throw` quando necessário.

Exemplo prático de RuntimeException

```
public class ExemploRuntimeException {  
    public static void main(String[] args) {  
        String texto = null;  
        int[] numbers = {1, 2, 3};  
        System.out.println(texto.length()); // Lança NullPointerException  
        System.out.println(5 / 0); // Lança ArithmeticException  
        System.out.println(numbers[3]); // Lança ArrayIndexOutOfBoundsException  
    }  
}
```

Tratamento de exceções e erros

Definições:

- **Tratamento de erros** é o processo de capturar e responder adequadamente a exceções
- Substitui códigos de retorno tradicionais por um **mecanismo estruturado**
- Permite **propagação automática** de erros através da pilha de chamadas
- **Throw**: lança uma exceção quando detecta uma condição anômala
- **Catch**: captura e trata exceções específicas
- Melhora **robustez** e **manutenibilidade** do código

Blocos try-catch-finally

Definições:

- **Try:** bloco onde se coloca código que pode gerar exceções
- **Catch:** bloco que captura e trata exceções específicas - pode haver múltiplos catches
- **Finally:** bloco que **sempre executa**, independente de exceções (usado para limpeza)
- **Try-with-resources:** sintaxe especial para recursos que implementam `AutoCloseable`
- Permite **recuperação** de erros ou **terminação controlada**
- **Ordem dos catches:** do mais específico para o mais genérico

Sintaxe básica do tratamento de exceções

```
try {  
    // Código que pode lançar exceções  
} catch (TipoDeExcecao1 e1) {  
    // Tratamento para TipoDeExcecao1  
} catch (TipoDeExcecao2 e2) {  
    // Tratamento para TipoDeExcecao2  
} finally {  
    // Código que sempre será executado  
}
```

Exmplo em outras linguagens:

- Python: try, except, finally
- C#: try, catch, finally
- JavaScript: try, catch, finally
- Ruby: begin, rescue, ensure
- C++: try, catch (com suporte limitado a exceções)

Tratamento de exceções em Java

```
try {  
    // Código que pode lançar exceções  
} catch (TipoDeExcecao1 e1) {  
    // Tratamento para TipoDeExcecao1  
} catch (Exception e2) {  
    // Tratamento para Exception  
}
```

A orientação a objetos traz benefícios significativos para o tratamento de exceções:

- Exceções são representadas por objetos que estendem a classe `Throwable`
- Permite capturar exceções específicas usando polimorfismo
- Exceções mais específicas devem ser capturadas antes das mais genéricas

Exercício – Exemplo de Exceção verificada

```
public static void main(String[] args) {  
    try {  
        FileReader file = new FileReader("arquivo.txt");  
        BufferedReader fileInput = new BufferedReader(file);  
        String line = fileInput.readLine();  
        System.out.println(line);  
        fileInput.close();  
    } catch (FileNotFoundException e) {  
        System.out.println("Arquivo não encontrado: " + e.getMessage());  
    } catch (IOException e) {  
        System.out.println("Erro ao ler o arquivo: " + e.getMessage());  
    }  
}
```

Exceções personalizadas

Definições:

- **Exceções personalizadas** são classes criadas para representar erros específicos da aplicação
- Devem **estender Exception** (checked) ou **RuntimeException** (unchecked)
- Permitem criar **semântica específica** para os erros do domínio
- Facilitam **tratamento diferenciado** por tipo de erro
- Devem ter **construtores adequados** (mensagem, causa, etc.)
- Melhoram **legibilidade** e **manutenibilidade** do código de tratamento

Boas práticas de tratamento

Definições:

- **Não ignore exceções:** sempre trate ou propague adequadamente
- **Seja específico:** capture exceções específicas, não genéricas (Exception)
- **Falhe rápido:** detecte erros o mais cedo possível
- **Mensagens informativas:** forneça contexto útil para depuração
- **Cleanup:** sempre libere recursos (use try-with-resources quando possível)

Exercício – Exceções personalizadas

Você deve criar uma exceção personalizada, como

`EstoqueInsuficienteException`, para tratar situações específicas no seu sistema de gerenciamento de estoque.

- Alterar o código da aula anterior para lançar e capturar essa exceção personalizada quando a quantidade em estoque for insuficiente.

Referências

- Page-Jones, M. *Fundamentos do Desenho Orientado a Objeto com UML*. Makron Books.
- Larman, C. *Utilizando UML e Padrões*. Saraiva.
- Ementa da disciplina.